

- [Login Module PHPUnit Test Documentation](#)
 - [Table of Contents](#)
 - [Overview](#)
 - [Project Structure](#)
 - [Test Environment Setup](#)
 - [PHPUnit Configuration \(phpunit.xml\)](#)
 - [Bootstrap File \(bootstrap.php\)](#)
 - [Mock Auth Class \(MockAuth.php\)](#)
 - [Constructor](#)
 - [Login Method](#)
 - [Logout Method](#)
 - [IsLoggedIn Method](#)
 - [Test Cases \(LoginTest.php\)](#)
 - [Test Class Setup](#)
 - [Test Empty Credentials](#)
 - [Test Invalid Credentials](#)
 - [Test Successful Login](#)
 - [Test Inactive User Login](#)
 - [Test Logout](#)
 - [Test Login Status](#)
 - [Running the Tests](#)
 - [Test Coverage Summary](#)
 - [Best Practices Implemented](#)

Login Module PHPUnit Test Documentation

Table of Contents

1. [Overview](#)
2. [Project Structure](#)
3. [Test Environment Setup](#)
4. [Mock Auth Class](#)
5. [Test Cases](#)
6. [Running the Tests](#)

Overview

This document provides a detailed explanation of the PHPUnit tests implemented for the login module of the Classroom application. The tests ensure that the authentication system works correctly for various scenarios including successful login, failed attempts, and edge cases.

Project Structure

```
tests/  
├─ bootstrap.php      # Test environment initialization  
├─ MockAuth.php       # Mock authentication class for testing  
└─ LoginTest.php      # Login test cases  
phpunit.xml           # PHPUnit configuration
```

Test Environment Setup

PHPUnit Configuration (phpunit.xml)

```
<?xml version="1.0" encoding="UTF-8"?>  
<phpunit bootstrap="tests/bootstrap.php"  
    colors="true"  
    verbose="true"  
    stopOnFailure="false">  
    <testsuites>  
        <testsuite name="Classroom Test Suite">  
            <directory>tests</directory>  
        </testsuite>  
    </testsuites>  
    <coverage processUncoveredFiles="true">  
        <include>  
            <directory suffix=".php">includes</directory>  
        </include>  
    </coverage>  
</phpunit>
```

Explanation:

- `bootstrap="tests/bootstrap.php"`: Specifies the initialization file for tests
- `colors="true"`: Enables colored output in test results
- `verbose="true"`: Provides detailed test execution information
- `stopOnFailure="false"`: Continues running tests even if one fails
- Coverage configuration includes all PHP files in the `includes` directory

Bootstrap File (bootstrap.php)

```
<?php
// Mock database configuration for testing
$db = new PDO(
    'sqlite::memory:',
    null,
    null,
    [PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION]
);

// Create test tables
$db->exec('
    CREATE TABLE IF NOT EXISTS users (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        username TEXT UNIQUE,
        email TEXT UNIQUE,
        password TEXT,
        full_name TEXT,
        role TEXT,
        is_active INTEGER DEFAULT 1
    )
');
```

Explanation:

- Creates an in-memory SQLite database for testing
- Sets up the database error mode to throw exceptions
- Creates a users table with the required schema
- Using SQLite in-memory database ensures tests are isolated and fast

Mock Auth Class (MockAuth.php)

```
<?php

class Auth {
    private $db;
```

```

private $useSession;

public function __construct($db = null, $useSession = true) {
    if ($db === null) {
        global $db;
        $this->db = $db;
    } else {
        $this->db = $db;
    }
    $this->useSession = $useSession;

    if ($useSession && session_status() !== PHP_SESSION_ACTIVE) {
        @session_start();
    }
}

public function login($username, $password) {
    if (empty($username) || empty($password)) {
        return ['success' => false, 'message' => 'Username and password are
required'];
    }

    // Find user by username or email
    $stmt = $this->db->prepare("SELECT * FROM users WHERE (username = ? OR
email = ?) AND is_active = 1");
    $stmt->execute([$username, $username]);
    $user = $stmt->fetch(PDO::FETCH_ASSOC);

    if (!$user || !password_verify($password, $user['password'])) {
        return ['success' => false, 'message' => 'Invalid credentials'];
    }

    if ($this->useSession) {
        // Set session variables
        $_SESSION['user_id'] = $user['id'];
        $_SESSION['username'] = $user['username'];
        $_SESSION['full_name'] = $user['full_name'];
        $_SESSION['role'] = $user['role'];
        $_SESSION['email'] = $user['email'];
    }

    return ['success' => true, 'message' => 'Login successful', 'user' =>
$user];
}

public function logout() {
    if ($this->useSession) {
        session_unset();
        session_destroy();
    }
    return ['success' => true, 'message' => 'Logged out successfully'];
}

public function isLoggedIn() {
    return $this->useSession && isset($_SESSION['user_id']);
}
}

```

Explanation by Method:

Constructor

```
public function __construct($db = null, $useSession = true)
```

- Takes optional database connection and session flag
- Allows dependency injection for testing
- Controls session handling through `$useSession` parameter
- Initializes session if needed and sessions are enabled

Login Method

```
public function login($username, $password)
```

- Validates input parameters
- Queries database for user by username or email
- Verifies password using secure hashing
- Manages session data when sessions are enabled
- Returns structured response with success status, message, and user data

Logout Method

```
public function logout()
```

- Cleans up session data if sessions are enabled
- Returns success confirmation
- Safe to call even when sessions are disabled

IsLoggedIn Method

```
public function isLoggedIn()
```

- Checks if user is currently logged in
- Only returns true if sessions are enabled and user_id exists in session
- Simple boolean return value

Test Cases (LoginTest.php)

Test Class Setup

```
<?php

use PHPUnit\Framework\TestCase;

require_once __DIR__ . '/MockAuth.php';

class LoginTest extends TestCase
{
    private $auth;
    private $db;

    protected function setUp(): void
    {
        // Create a mock PDO object
        $this->db = $this->createMock(PDO::class);

        // Create the Auth instance with mocked database and disabled sessions
        $this->auth = new Auth($this->db, false);
    }
}
```

Explanation:

- Extends PHPUnit's TestCase class
- Sets up test dependencies in setUp method
- Creates mock PDO object for database operations
- Initializes Auth class with mocked database and disabled sessions

Test Empty Credentials

```
public function testLoginWithEmptyCredentials()  
{  
    $result = $this->auth->login('', '');  
    $this->assertFalse($result['success']);  
    $this->assertEquals('Username and password are required', $result['message']);  
}
```

Explanation:

- Tests input validation
- Ensures empty credentials are rejected
- Verifies appropriate error message
- Checks success flag is false

Test Invalid Credentials

```
public function testLoginWithInvalidCredentials()  
{  
    // Mock the database query for invalid credentials  
    $stmt = $this->createMock(PDOStatement::class);  
    $stmt->method('fetch')->willReturn(false);  
    $stmt->method('execute')->willReturn(true);  
  
    $this->db->method('prepare')->willReturn($stmt);  
  
    $result = $this->auth->login('invalid_user', 'wrong_password');  
    $this->assertFalse($result['success']);  
    $this->assertEquals('Invalid credentials', $result['message']);  
}
```

Explanation:

- Mocks database query returning no user
- Tests login attempt with non-existent user
- Verifies appropriate error message
- Ensures security by not revealing specific failure reason

Test Successful Login

```
public function testSuccessfulLogin()  
{
```

```

// Mock user data
$userData = [
    'id' => 1,
    'username' => 'testuser',
    'password' => password_hash('correct_password', PASSWORD_DEFAULT),
    'email' => 'test@example.com',
    'full_name' => 'Test User',
    'role' => 'student',
    'is_active' => 1
];

// Mock the database query for valid credentials
$stmt = $this->createMock(PDOStatement::class);
$stmt->method('fetch')->willReturn($userData);
$stmt->method('execute')->willReturn(true);

$this->db->method('prepare')->willReturn($stmt);

$result = $this->auth->login('testuser', 'correct_password');
$this->assertTrue($result['success']);
$this->assertEquals('Login successful', $result['message']);
$this->assertEquals($userData, $result['user']);
}

```

Explanation:

- Creates mock user data with hashed password
- Simulates successful database query
- Tests successful login scenario
- Verifies user data is returned correctly
- Checks success message and flag

Test Inactive User Login

```

public function testLoginWithInactiveUser()
{
    // Mock inactive user data
    $userData = [
        'id' => 2,
        'username' => 'inactive_user',
        'password' => password_hash('password', PASSWORD_DEFAULT),
        'email' => 'inactive@example.com',
        'full_name' => 'Inactive User',
        'role' => 'student',
        'is_active' => 0
    ];

    // Mock the database query
    $stmt = $this->createMock(PDOStatement::class);
}

```



```
$stmt->method('fetch')->willReturn(false);
$stmt->method('execute')->willReturn(true);

$this->db->method('prepare')->willReturn($stmt);

$result = $this->auth->login('inactive_user', 'password');
$this->assertFalse($result['success']);
$this->assertEquals('Invalid credentials', $result['message']);
}
```

Explanation:

- Tests login attempt with inactive user account
- Verifies inactive users cannot log in
- Ensures security by not revealing account status
- Checks appropriate error response

Test Logout

```
public function testLogout()
{
    $result = $this->auth->logout();
    $this->assertTrue($result['success']);
    $this->assertEquals('Logged out successfully', $result['message']);
}
```

Explanation:

- Tests logout functionality
- Verifies successful logout response
- Ensures clean session termination
- Checks appropriate success message

Test Login Status

```
public function testIsLoggedIn()
{
    $this->assertFalse($this->auth->isLoggedIn());
}
```

Explanation:

- Tests login status check functionality
- Verifies default not-logged-in state
- Simple boolean assertion

Running the Tests

To run the tests, use the following command in the project root directory:

```
vendor/bin/phpunit
```

Expected output should show all tests passing:

```
PHPUnit 9.6.29 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.4.10
Configuration: D:\xampp\htdocs\Classroom\phpunit.xml

.....                                                6 / 6 (100%)

Time: 00:00.975, Memory: 6.00 MB

OK (6 tests, 12 assertions)
```

Test Coverage Summary

The test suite covers:

1. Input Validation
 - Empty credentials
 - Invalid credentials
2. Authentication Logic
 - Successful login
 - Failed login
 - Inactive user handling
3. Session Management
 - Login session creation

- Logout functionality
- Login status checking

4. Security

- Password verification
- Account status checking
- Generic error messages

Best Practices Implemented

1. Dependency Injection

- Database connection is injectable for testing
- Session handling can be disabled for testing

2. Mocking

- Database connections are mocked
- PDO statements are mocked
- User data is mocked

3. Isolation

- Tests run independently
- No real database connections
- No real session handling

4. Security

- Password hashing
- Generic error messages
- Active account verification

5. Clean Code

- Clear test names
- Organized test structure
- Comprehensive assertions